

Universidad de San Carlos de Guatemala

Division de las Ciencias de la Ingenieria

Ingenieria en Sistemas

Organizacion De Lenguajes Y Compiladores 1

Seccion: A

Auxiliar: Carlos Fernando Enrique Lopez Garcia



“Manual Tecnico Proyecto Golite Fase 1”

Estudiante:

Mynor Miguel Monzón Martínez 3195097021306

Guatemala, 12 de junio de

2026

1. Introduccion

El presente documento es el Manual Tecnico del Interprete GoLite, desarrollado como Fase 1 del proyecto del curso Organizacion de Lenguajes y Compiladores 1. El interprete esta implementado en Java, utilizando JFlex para el analisis lexico y CUP para el analisis sintactico, con una interfaz grafica construida en Java Swing.

GoLite es un subconjunto del lenguaje Go disenado para explorar los conceptos fundamentales de compiladores: analisis lexico, analisis sintactico, analisis semantico y recorrido del Arbol de Sintaxis Abstracta (AST). En esta Fase 1 se implementan los tipos de datos primitivos, variables, operadores, sentencias de control de flujo, funciones basicas y funciones embebidas.

2. Arquitectura del Sistema

2.1 Estructura de paquetes

El proyecto sigue una arquitectura en capas, organizada bajo Maven con la siguiente estructura de paquetes:

```
com.golite
  lexer/          -> Lexer.java   (generado por JFlex desde
lexer.flex)
  parser/         -> parser.java + Sym.java   (generados por CUP
desde parser.cup)
  ast/
    NodoAST.java          -> Clase base abstracta del AST
    expresiones/         -> Nodos de expresiones (literales,
operadores, id)
    sentencias/          -> Nodos de sentencias (if, for,
declaracion, etc.)
    interpreter/
      Interprete.java      -> Singleton principal de ejecucion
      Entorno.java         -> Manejo de scopes / tabla de
simbolos
      ErrorSemantic.java   -> Modelo de error
    gui/
      VentanaPrincipal.java -> Interfaz grafica Swing
```

2.2 Flujo de ejecucion

El flujo de procesamiento de un archivo .glt sigue las etapas que se describen a continuacion:

Etapas	Componente	Descripcion
1	JFlex / Lexer	Tokenizacion del codigo fuente. Reconoce lexemas, genera objetos Symbol para CUP y registra cada token con su posicion.
2	CUP / Parser	Analisis sintactico LALR(1). Valida la estructura gramatical y construye el AST como una lista de objetos NodoAST.

Etapas	Componente	Descripcion
3	Interprete	Recorre el AST en profundidad. Ejecuta cada nodo evaluando expresiones y aplicando las sentencias correspondientes.
4	Entorno	Gestiona los ambitos anidados mediante una pila de tablas de simbolos (HashMap). Resuelve y almacena variables.
5	GUI Swing	Presenta el editor, la consola de salida, la tabla de tokens y el reporte de errores al usuario.

3. Analizador Lexico (JFlex)

3.1 Descripcion general

El analizador lexico esta definido en el archivo lexer.flex y es generado automaticamente por JFlex 1.9.1. Implementa la interfaz requerida por CUP mediante las directivas %cup y %cupsym. Cada vez que reconoce un lexema, registra el token en el Interprete singleton para el reporte de tokens y devuelve un objeto Symbol con la posicion exacta de linea y columna.

3.2 Definiciones de expresiones regulares

Las siguientes macros definen los patrones basicos que usa el lexer:

Macro	Expresion Regular	Descripcion
LineTerminator	<code>\r \n \r\n</code>	Fin de linea
WhiteSpace	<code>{LineTerminator} [\t\f]</code>	Espacios en blanco (ignorados)
Digit	<code>[0-9]</code>	Digito decimal
Letter	<code>[a-zA-Z_]</code>	Letra o guion bajo
Identifier	<code>{Letter}({Letter} {Digit})*</code>	Identificador valido
IntLiteral	<code>{Digit}+</code>	Literal entero
FloatLiteral	<code>{Digit}+"." {Digit}+</code>	Literal flotante de 64 bits
StringLiteral	<code>"([^\\"\\n] \\.)*"</code>	Cadena entre comillas dobles
RuneLiteral	<code>'([^\\"\\n] \\.)*'</code>	Caracter rune entre comillas simples

3.3 Tokens reconocidos

A continuacion se detallan los grupos de tokens que el lexer es capaz de reconocer.

Palabras reservadas:

Lexema	Token	Descripcion
var	VAR	Declaracion de variable con tipo explicito
func	FUNC	Declaracion de funcion
main	MAIN	Funcion principal de entrada al programa
if	IF	Inicio de sentencia condicional
else	ELSE	Rama alternativa del if
for	FOR	Bucle iterativo
break	BREAK	Interrumpe el bucle actual
continue	CONTINUE	Pasa a la siguiente iteracion del bucle
return	RETURN	Retorno de funcion (reservado para Fase 2)
true	TRUE	Literal booleano verdadero
false	FALSE	Literal booleano falso
nil	NIL	Valor nulo

Tipos de datos:

Lexema	Token	Tipo en GoLite
int	TINT	Entero de 32 bits con signo
float64	TFLOAT64	Numero de punto flotante IEEE 754 de 64 bits
string	TSTRING	Cadena de caracteres
bool	TBOOL	Valor booleano (true o false)
rune	TRUNE	Caracter Unicode (alias de uint8)

Funciones embebidas:

Lexema	Token	Descripcion
fmt.Println	PRINTLN	Impresion en consola con salto de linea automatico
strconv.Atoi	atoi	Conversion de cadena a entero
strconv.ParseFloat	PARSEFLOAT	Conversion de cadena a flotante de 64 bits
reflect.TypeOf	TYPEOF	Obtiene el tipo de una expresion en tiempo

Lexema	Token	Descripcion
		de ejecucion

Operadores:

Lexema	Token	Descripcion
+	PLUS	Suma o concatenacion de cadenas
-	MINUS	Resta o negacion unaria
*	TIMES	Multiplicacion
/	DIV	Division
%	MOD	Modulo (solo entre enteros)
++	INC	Incremento en uno (sintactico para += 1)
--	DEC	Decremento en uno (sintactico para -= 1)
=	ASSIGN	Asignacion simple
:=	DECL_ASSIGN	Declaracion con inferencia de tipo
+=	PLUS_ASSIGN	Asignacion con suma
-=	MINUS_ASSIGN	Asignacion con resta
==	EQ	Igualdad
!=	NEQ	Desigualdad
<	LT	Menor que
>	GT	Mayor que
<=	LEQ	Menor o igual que
>=	GEQ	Mayor o igual que
&&	AND	Operador logico AND
	OR	Operador logico OR
!	NOT	Negacion logica

3.4 Manejo de errores lexicos

Cualquier caracter que no coincida con ninguna de las reglas anteriores activa la regla catch-all [^], que registra un error lexico en el Interpretador con el caracter no reconocido, la linea y la columna exactas. El analisis continua sin interrumpirse, lo que permite detectar multiples errores en una sola pasada del codigo fuente.

4. Gramatica del Lenguaje GoLite en Notacion BNF

4.1 Descripcion general

La gramatica esta definida en el archivo parser.cup y es procesada por CUP para generar un analizador sintactico LALR(1). La gramatica es no ambigua gracias al uso de llaves obligatorias en todos los bloques y a la declaracion explicita de precedencia y asociatividad de los operadores.

Se utiliza la siguiente convencion en la notacion BNF:

- Los no terminales se escriben en minusculas con guion bajo (ej. stmt_list).
- Los terminales se escriben en mayusculas (ej. IF, LBRACE).
- El simbolo ::= separa la cabeza de la produccion de su cuerpo.
- El simbolo | separa alternativas de una misma produccion.
- La palabra epsilon representa la produccion vacia (cadena vacia).

4.2 Producciones de la gramatica

Simbolo inicial:

```
<program> ::= FUNC MAIN LPAREN RPAREN LBRACE <stmt_list> RBRACE
```

El programa completo es una unica funcion main. El interprete la localiza como punto de entrada y ejecuta sus sentencias de forma secuencial.

Lista de sentencias:

```
<stmt_list> ::= <stmt_list> <stmt>
               | epsilon
```

La lista de sentencias es recursiva por la izquierda, lo cual es eficiente para parsers LALR. Una lista vacia es valida, por lo que un bloque puede no contener sentencias.

Sentencia:

```
<stmt> ::= <decl_stmt>    <opt_semi>
          | <assign_stmt> <opt_semi>
          | <if_stmt>
          | <for_stmt>
          | <print_stmt>  <opt_semi>
          | BREAK         <opt_semi>
          | CONTINUE      <opt_semi>
          | LBRACE <stmt_list> RBRACE
```

```
<opt_semi> ::= SEMICOLON
             | epsilon
```

El punto y coma es opcional. La produccion `opt_semi` permite ambos estilos sin generar ambigüedad porque el parser LALR resuelve el lookahead correctamente. El ultimo caso (bloque anonimo) permite declarar bloques independientes con su propio ambito.

Declaracion de variables:

```
<decl_stmt> ::= VAR ID <tipo> ASSIGN <expr>
              | VAR ID <tipo>
              | ID DECL_ASSIGN <expr>
```

Hay tres formas de declarar variables: declaracion explicita con tipo y valor inicial (`var x int = 10`), declaracion explicita con valor por defecto (`var x int`), y declaracion implicita con inferencia de tipo (`x := 10`). Una vez asignado el tipo, no puede cambiar durante la ejecucion.

Tipos de datos:

```
<tipo> ::= TINT | TFLOAT64 | TSTRING | TBOOL | TRUNE
```

Asignacion:

```
<assign_stmt> ::= ID ASSIGN expr
                | ID PLUS_ASSIGN expr
                | ID MINUS_ASSIGN expr
                | ID INC
                | ID DEC
```

Los operadores `++` y `--` son azucar sintactico. El interprete los convierte internamente en `NodoAsignacion` con operador `+=` o `-=` y un `NodoEntero(1)`, evitando duplicar logica de evaluacion.

Sentencia if-else:

```
<if_stmt> ::= IF <expr> LBRACE <stmt_list> RBRACE
            | IF <expr> LBRACE <stmt_list> RBRACE ELSE LBRACE
<stmt_list> RBRACE
            | IF <expr> LBRACE <stmt_list> RBRACE ELSE <if_stmt>
```

La tercera produccion maneja el else-if de forma recursiva. No existe el problema del dangling else porque GoLite exige llaves en todos los bloques, lo que hace que el parser no tenga ambigüedad en esta regla.

Sentencia for:

```
<for_stmt> ::= FOR <expr> LBRACE <stmt_list> RBRACE
            | FOR <decl_stmt> SEMICOLON <expr> SEMICOLON
<assign_stmt>
    LBRACE <stmt_list> RBRACE
```

La primera produccion equivale al while de otros lenguajes. La segunda es el for clasico con inicializacion, condicion e incremento separados por punto y coma. El for range queda excluido de la Fase 1.

Sentencia de impresion:

```
<print_stmt> ::= PRINTLN LPAREN <expr_list> RPAREN  
               | PRINTLN LPAREN RPAREN
```

fmt.Println sin argumentos imprime unicamente un salto de linea. Con argumentos, imprime cada expresion separada por un espacio y finaliza con salto de linea.

Lista de expresiones:

```
<expr_list> ::= <expr_list> COMMA <expr>  
               | <expr>
```

Expresiones:

```
<expr> ::= <expr> PLUS      <expr>      // suma o concatenacion de  
strings  
         | <expr> MINUS    <expr>      // resta  
         | <expr> TIMES    <expr>      // multiplicacion  
         | <expr> DIV      <expr>      // division  
         | <expr> MOD      <expr>      // modulo (int % int solamente)  
         | <expr> EQ       <expr>      // igualdad  
         | <expr> NEQ      <expr>      // desigualdad  
         | <expr> LT       <expr>      // menor que  
         | <expr> GT       <expr>      // mayor que  
         | <expr> LEQ      <expr>      // menor o igual que  
         | <expr> GEQ      <expr>      // mayor o igual que  
         | <expr> AND      <expr>      // logico AND  
         | <expr> OR       <expr>      // logico OR  
         | NOT      <expr>      // negacion logica  
         | MINUS    <expr>      // negacion unaria [%prec  
UMINUS]  
         | LPAREN  <expr> RPAREN    // agrupacion con parentesis  
         | INT_LIT  
         | FLOAT_LIT  
         | STRING_LIT  
         | TRUE   | FALSE   | NIL   | RUNE_LIT  
         | ID  
         | ATOI      LPAREN <expr> RPAREN  
         | PARSEFLOAT LPAREN <expr> RPAREN  
         | TYPEOF LPAREN <expr> RPAREN DOT ID LPAREN RPAREN
```

4.3 Precedencia y asociatividad de operadores

La siguiente tabla muestra los niveles de precedencia declarados en el archivo CUP, de menor a mayor prioridad. CUP usa esta informacion para resolver conflictos shift/reduce en la gramatica LALR(1) sin necesidad de escribir reglas gramaticales adicionales.

Nivel	Operadores	Asociatividad	Descripcion
1 (menor)		Izquierda	OR logico

Nivel	Operadores	Asociatividad	Descripcion
2	&&	Izquierda	AND logico
3	== !=	Izquierda	Igualdad y desigualdad
4	< > <= >=	Izquierda	Operadores relacionales
5	+ - (binario)	Izquierda	Suma y resta
6	* / %	Izquierda	Multiplicacion, division, modulo
7 (mayor)	! - (UMINUS)	Derecha	Negacion logica y unaria

La directiva %prec UMINUS en la produccion de negacion unaria resuelve el conflicto de que el token MINUS es compartido entre la resta binaria y la negacion unaria. Al asignar una precedencia especial al nodo de negacion, CUP sabe que -a debe tener mayor prioridad que a - b.

4.4 Demostracion de no-ambigüedad

La gramatica es no ambigua por los siguientes motivos:

- Bloques obligatorios: if, else y for siempre requieren llaves. Esto elimina el problema del dangling else que ocurre en lenguajes como C donde las llaves son opcionales.
- Precedencia declarativa: Los conflictos shift/reduce en expresiones binarias se resuelven mediante la tabla de precedencia de CUP, garantizando que $2 + 3 * 4$ se evalúe como $2 + (3 * 4)$ y que $1 == 2 \&\& 3 < 4$ se evalúe como $(1 == 2) \&\& (3 < 4)$.
- Separacion de stmt y expr: Las sentencias no pueden aparecer dentro de una expresion y viceversa, eliminando ambigüedades entre ambos contextos.
- opt_semi sin ambigüedad: El punto y coma opcional se maneja con una produccion separada que CUP resuelve con un lookahead de un simbolo.
- UMINUS con %prec: La directiva evita el conflicto del operador MINUS compartido entre resta binaria y negacion unaria.

5. Arbol de Sintaxis Abstracta (AST)

5.1 Clase base NodoAST

Todos los nodos del AST heredan de la clase abstracta NodoAST, que define el metodo abstracto ejecutar(Entorno entorno). Este disenio implementa un patron de visitor implicito: cada nodo sabe como ejecutarse a si mismo recibiendo el entorno actual para resolver variables.

5.2 Nodos de expresion

Los nodos de expresion son subclases que evaluan y retornan un valor:

Clase	Descripcion	Atributos clave
NodoEntero	Literal int	int valor
NodoDecimal	Literal float64	double valor
NodoString	Literal string	String valor
NodoBooleano	Literal bool	boolean valor
NodoRune	Literal rune	char valor
NodoIdentificador	Acceso a variable por nombre	String nombre
NodoSuma	Suma aritmetica o concatenacion de cadenas	NodoAST izq, der
NodoResta	Resta aritmetica	NodoAST izq, der
NodoMultiplicacion	Multiplicacion	NodoAST izq, der
NodoDivision	Division con verificacion de cero	NodoAST izq, der
NodoModulo	Modulo int % int	NodoAST izq, der
NodoComparacion	Operadores ==, !=, <, >, <=, >=	NodoAST izq, String op, der
NodoLogico	Operadores &&, , !	NodoAST izq, String op, der
NodoNegacion	Negacion unaria -expr	NodoAST expr
NodoAtoi	strconv.Atoi(expr)	NodoAST expr
NodoParseFloat	strconv.ParseFloat(expr)	NodoAST expr
NodoTypeOf	reflect.TypeOf(expr).String()	NodoAST expr

5.3 Nodos de sentencia

Clase	Descripcion	Atributos clave
NodoPrograma	Raiz del AST. Contiene la lista de sentencias de main	List<NodoAST> sentencias
NodoDeclaracion	Declaracion de variable (var o :=)	String nombre, tipo; NodoAST valor
NodoAsignacion	Asignacion o asignacion compuesta (=, +=, -=)	String nombre, op; NodoAST expr
NodoIf	Condicional if / else if / else	NodoAST cond; List then, else_; NodoIf elseIf
NodoFor	Bucle for (while o clasico con init/cond/update)	NodoAST cond, init; NodoAsig update; List cuerpo

Clase	Descripcion	Atributos clave
NodoBloque	Bloque anonimo con su propio ambito	List<NodoAST> sentencias
NodoPrintln	fmt.Println con lista de argumentos	List<NodoAST> args
NodoBreak	Interrumpe el bucle actual mediante excepcion interna	linea, columna
NodoContinue	Pasa a la siguiente iteracion mediante excepcion interna	linea, columna

6. Interprete y Gestion de Entornos

6.1 Clase Interprete

El Interprete es un singleton que actua como controlador central. Sus responsabilidades son: invocar al lexer y al parser para generar el AST, llamar a NodoPrograma.ejecutar() para iniciar la ejecucion, mantener la lista de tokens reconocidos para el reporte, mantener la lista de errores de todas las fases, y proveer la salida de consola a la GUI.

6.2 Clase Entorno

La clase Entorno implementa el manejo de ambitos mediante una pila de HashMap, donde cada HashMap representa un bloque de sentencias. Sus operaciones principales son:

Metodo	Descripcion
pushBloque()	Crea un nuevo HashMap y lo apila. Se llama al entrar a un bloque { }.
popBloque()	Elimina el HashMap del tope de la pila. Se llama al salir del bloque.
declararVariable(nombre, val)	Registra la variable en el HashMap del tope (scope actual).
obtenerVariable(nombre)	Busca la variable desde el tope hasta la base de la pila (scope lookup).
asignarVariable(nombre, val)	Modifica el valor en el scope donde fue declarada la variable.

Este diseno permite correctamente que una variable declarada en un bloque interno oculte a otra con el mismo nombre en un bloque externo (shadowing), y que al salir del bloque la variable interna desaparezca sin afectar a la variable del scope superior.

6.3 Manejo de break y continue

Las sentencias break y continue se implementan mediante excepciones internas de Java (BreakException y ContinueException). Cuando NodoBreak se ejecuta, lanza una BreakException que es capturada por NodoFor en su bloque catch. Este patron evita la necesidad de flags booleanos distribuidos en la logica de ejecucion.

6.4 Sistema de tipos y coercion implicita

El interprete aplica las siguientes reglas de coercion implicita al evaluar expresiones aritmeticas:

Operandos	Tipo resultante
int + int	int
int + float64	float64
float64 + int	float64
float64 + float64	float64
string + string	string (concatenacion)
int asignado a float64	conversion automatica a double

Cualquier otra combinacion de tipos genera un error semantico con descripcion detallada de la incompatibilidad.

7. Interfaz Grafica (Java Swing)

7.1 Componentes de la GUI

Componente	Tipo Swing	Funcion
Editor de codigo	JTextArea / JScrollPane	Permite escribir y editar archivos .glt. Soporta apertura y guardado.
Consola de salida	JTextArea (solo lectura)	Muestra la salida de fmt.Println y mensajes del interprete.
Barra de menu	JMenuBar / JMenu	Acceso a Archivo (nuevo, abrir, guardar) y Ejecutar.
Tabla de tokens	JTable	Reporte con No., Lexema, Tipo, Linea y Columna de cada token.
Reporte de errores	JTable	Reporte con No., Descripcion, Linea, Columna y Tipo de error.

7.2 Flujo de uso desde la GUI

El usuario escribe o abre un archivo .glt en el editor y presiona Ejecutar. La GUI invoca al Interprete singleton, que realiza el analisis lexico, sintactico y semantico. La consola muestra la salida generada por `fmt.Println`, la tabla de tokens se actualiza con todos los tokens reconocidos y el reporte de errores muestra todos los errores encontrados en cualquier fase.

8. Herramientas y Configuracion del Proyecto

Herramienta	Version	Uso en el proyecto
Java (JDK)	17 o superior	Lenguaje de implementacion. Logica del interprete, AST y GUI.
JFlex	1.9.1	Generador del analizador lexico a partir de <code>lexer.flex</code> .
CUP	11b-20160615	Generador del analizador sintactico LALR(1) a partir de <code>parser.cup</code> .
Apache Maven	3.9.x	Gestion de dependencias, compilacion y generacion del JAR ejecutable.
Java Swing	Incluido en JDK	Framework de interfaz grafica de usuario.
IntelliJ IDEA	Community Ed.	Entorno de desarrollo utilizado.
Ubuntu	Linux	Sistema operativo de desarrollo.

8.1 Construcccion del proyecto

El proyecto se compila con Maven desde el directorio `GoLiteInterpreter/`:

```
cd GoLiteInterpreter/  
mvn clean package
```

Maven ejecuta los plugins de JFlex y CUP durante la fase `generate-sources`, generando `Lexer.java`, `parser.java` y `Sym.java` antes de compilar el resto del codigo. El resultado final es un JAR ejecutable con todas las dependencias incluidas.

8.2 Ejecucion

```
java -jar target/GoLiteInterpreter-1.0-SNAPSHOT.jar
```

9. Conclusiones

- El interprete GoLite Fase 1 implementa correctamente todos los componentes requeridos: analisis lexico con JFlex, analisis sintactico LALR(1) con CUP, analisis semantico y ejecucion mediante recorrido del AST.
- La gramatica definida en parser.cup es no ambigua gracias al uso de llaves obligatorias en todos los bloques y a la declaracion explicita de precedencia y asociatividad de operadores en CUP.
- El manejo de ambitos mediante pila de HashMap en la clase Entorno garantiza el correcto comportamiento del shadowing de variables y la limpieza automatica de variables locales al salir de su bloque.
- El uso de excepciones internas para break y continue mantiene el codigo del interprete limpio y libre de flags globales de estado.
- La arquitectura en capas facilita la extension del proyecto en la Fase 2, donde se implementaran slices, structs, funciones con retorno, switch-case y el for range.